UNIWERSYTET WARMIŃSKO-MAZURSKI W OLSZTYNIE

Programowanie deklaratywne - paradygmaty programowania

Laboratorium 08

Podstawy programowania w Haskell

Krotka to jedna z prostych struktur danych w Haskellu. Służy do przechowywania kilku wartości razem.

Krotka jest:

- uporządkowana — kolejność elementów ma znaczenie,
- może przechowywać wartości różnych typów,
- ma stałą długość — po utworzeniu nie możemy „dodać” kolejnego elementu tak jak do listy.

Przykłady krotek:

```
(1, 2) -- (Int, Int)

("Ala", 20) -- (String, Int)

("Lalka", "Bolesław Prus", 1890) -- (String, String, Int)

(True, "OK") -- (Bool, String)
```

W przeciwieństwie do listy, krotka może zawierać elementy różnych typów.

```
[1,2,3] -- lista liczb Int

("Ala", 20) -- krotka zawierająca String i Int
```

Do tworzenia krotek używamy nawiasów okrągłych i przecinków. Przykład:

```
student :: (String, Int)
student = ("Ala", 20)
```

Do pobierania elementów z par możemy użyć funkcji `fst` oraz `snd`.

```
fst ("Ala", 20) -- "Ala"

snd ("Ala", 20) -- 20
```

- Funkcja `fst` zwraca pierwszy element pary.
- Funkcja `snd` zwraca drugi element pary.

Częściej jednak używa się dopasowania wzorca, czyli pattern matchingu.

```
getName :: (String, Int) -> String
getName (name, age) = name

getAge :: (String, Int) -> Int
getAge (name, age) = age
```

Jeżeli jakiś element krotki nie jest nam potrzebny, możemy użyć `_`.

```
getName :: (String, Int) -> String
getName (name, _) = name

getAge :: (String, Int) -> Int
getAge (_, age) = age
```

Znak `_` oznacza, że dana wartość istnieje, ale nie będziemy jej używać.

Możemy też definiować własne nazwy typów za pomocą `type`.

```
type Student = (String, Int)

student1 :: Student
student1 = ("Ala", 20)
```

Przykład funkcji sprawdzającej, czy student jest pełnoletni:

```
type Student = (String, Int)

isAdult :: Student -> Bool
isAdult (_, age) = age >= 18

isAdult ("Ala", 20) -- True

isAdult ("Bartek", 17) -- False
```

Krotki często pojawiają się razem z listami. Możemy mieć listę studentów:

```
students :: [(String, Int)]
students = [("Ala", 20), ("Bartek", 17), ("Celina", 21)]
```

Przydatną funkcją jest `zip`, która łączy dwie listy w listę par.

```
zip [1,2,3] ["a","b","c"]
-- [(1,"a"),(2,"b"),(3,"c")]

names = ["Ala", "Bartek", "Celina"]
ages = [20, 17, 21]

zip names ages
-- [("Ala",20),("Bartek",17),("Celina",21)]
```

Funkcja `unzip` działa odwrotnie.

```
unzip [("Ala",20),("Bartek",17)]
-- (["Ala","Bartek"],[20,17])
```

Zadania

1. Napisz funkcję zamieniającą elementy pary miejscami.
2. Napisz funkcję przyjmującą dwa punkty i zwracające wektor.
3. Napisz funkcję dodającą dwa wektory.
4. Napisz funkcję sprawdzającą, czy punkt leży na osi X.
5. Napisz funkcję sprawdzającą, czy punkt leży na osi Y.
6. Napisz funkcję, która dla dwóch liczb zwraca parę: mniejsza liczba, większa liczba.
7. Zdefiniuj typ `Book` gdzie pierwszy element to tytuł, drugi to autor, a trzeci to rok wydania. Następnie napisz funkcje:

```
title :: Book -> String
author :: Book -> String
year :: Book -> Int
```

8. Napisz funkcję, która przyjmuje parę liczb i zwraca ich sumę.
9. Napisz funkcję, która przyjmuje parę liczb i zwraca ich iloczyn.